Reviewer Pack — Minimal Order of Brauer Characters and Frege Proof Depth Cor…

Entry 5b4276146432 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 01:17:01 UTC

Minimal Order of Brauer Characters and Frege Proof Depth Correlation

Entry ID: `5b4276146432` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 01:17:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory (Brauer Groups) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For every Boolean formula φ, the minimal order of a non-trivial character in the Brauer group of the associated vector space over the field with two elements is linearly correlated with its Frege proof depth, such that MinimalCharacterOrder(φ) = Θ(FregeProofDepth(φ)).

Rationale (proposer's reasoning):

> The Brauer group captures the algebraic structure of the vector space encoding the formula's variables and clauses. A deeper algebraic structure suggests a more complex interplay of logical operations, which might correlate with an increased proof depth.

Taxonomy category: `BrauerGroupToFregeProofComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d1b71f4a22f29491`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson's correlation coefficient between MinimalCharacterOrder and FregeProofDepth across 30 random seeds exceeds 0.5 and the p-value is less than 0.05, indicating a statistically significant linear relationship.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Character Order Brauer Group Boolean Satisfiability` - `Frege Proof Depth Correlation Algebraic K-Theory` - `Vector Space Two Elements Character Order Frege Complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order p^aq - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1007.3011v1] Counting lifts of Brauer characters - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/2104.04021v2] A universal characterization of noncommutative motives and secondary algebraic K-theory - [http://arxiv.org/abs/1811.09564v3] Dévissage for Waldhausen K-theory - [http://arxiv.org/abs/2402.10351v3] Quantum Automating $\mathbf{TC}^0$-Frege Is LWE-Hard - [http://arxiv.org/abs/1311.2501v4] A reduction of proof complexity to computational complexity for $AC^0[p]$ Frege systems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n_vars, n_clauses):
        variables = [f'x{i}' for i in range(n_vars)]
        clauses = []
        for _ in range(n_clauses):
            clause = random.choice(['', '¬']) + random.choice(variables) for _ in range(random.ran
            clauses.append('∨'.join(clause))
        return '∧'.join(clauses)

    def frege_proof_depth(formula):
        # Simplified estimation of Frege proof depth
        return len(formula.split('∧')) + len(formula.split('∨'))

    def minimal_brauer_character_order(n_vars, n_clauses):
        # Simplified estimation of minimal Brauer character order
        return max(n_vars, n_clauses)

    results = []
    for _ in range(30):
```

```
        n_vars = random.randint(5, 10)
        n_clauses = random.randint(5, 10)
        formula = generate_boolean_formula(n_vars, n_clauses)
        depth = frege_proof_depth(formula)
        order = minimal_brauer_character_order(n_vars, n_clauses)
        results.append((depth, order))

    if not results:
        return {
            "metric_name": "Pearson's Correlation Coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 10,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    depths = [r[0] for r in results]
    orders = [r[1] for r in results]

    n = len(depths)
    mean_depth = sum(depths) / n
    mean_order = sum(orders) / n

    covariance = sum((depths[i] - mean_depth) * (orders[i] - mean_order) for i in range(n)) / n
    variance_depth = sum((depths[i] - mean_depth) ** 2 for i in range(n)) / n
    variance_order = sum((orders[i] - mean_order) ** 2 for i in range(n)) / n

    correlation_coefficient = covariance / (math.sqrt(variance_depth) * math.sqrt(variance_order))

    return {
        "metric_name": "Pearson's Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": n,
        "n_max": 10,
        "conjecture_holds": abs(correlation_coefficient) > 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no trials executed")
        sys.exit(0)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'}
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.14799345005018424, 'instances_tested': 30,
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.2180643096186563, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.08939467008744187, 'instances_tested': 30,
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.15423405993794964, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.0851381455068878, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.0927286218647324, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.23534262159718713, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.007519090404594688, 'instances_tested': 30,
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.2871968025733203, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.031040194980848337, 'instances_tested': 30,
TRIAL: {'metric_name': 'Correlation', 'metric_value': 0.21272755940336624, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Correlation', 'metric_value': -0.0023146940535794264, 'instances_tested': 30
RESULT: FALSIFIED counterexample="" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code uses simplified estimations for both Frege proof depth and minimal Brauer character order, which are not faithful to the mathematical definitions. The Frege proof depth is estimated by counting logical operators, which does not capture the actual complexity of a proof. Minimal Brauer character order is estimated as the maximum of the number of variables and clauses, which is trivially incorrect for complex Boolean formulas.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The correlation coefficient between MinimalCharacterOrder and FregeProofDepth did not exceed the threshold of 0.5 across all seeds tested, and there w | next: Re-evaluate the conjecture using more accurate methods for calculating Frege proof depth and minimal Brauer character order. Consider exploring alternative metrics or approaches to validate the correlation.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18838
2	preregistration	ollama_remote	glm4:latest	0	0	9391
3	novelty	ollama_remote	glm4:latest	0	0	8148
4	novelty	ollama_remote	glm4:latest	0	0	9954
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	49204
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8631
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10016
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10561
9	critic	ollama_remote	glm4:latest	0	0	33265
10	judge	ollama_remote	glm4:latest	0	0	9728

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 167737 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5b4276146432.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5b4276146432.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5b4276146432.tar.gz` (if generated)